

SafeNav-LLM: A Layer-by-Layer Architectural Analysis

On-Device Semantic Waypoint Resolution for Nav2

Abstract

This document provides a layer-by-layer explanation of the SafeNav-LLM architecture: an on-device, closed-vocabulary, Nav2-native language model resolver for semantic navigation goals. The problem is not simply “using an LLM for navigation,” but making semantic goal resolution safe, offline, and native to Nav2. Four architectural gaps are examined in depth — on-device inference, closed-set structural safety, native BehaviorTree integration, and deployment-calibrated confidence — along with how they compose into a single defense-in-depth system.

Contents

1	Understanding the Problem	3
2	Gap A — Fully On-Device LLM Inference	3
2.1	Cloud-Based Architecture and Its Dependencies	3
2.2	Privacy Dependency	3
2.3	Latency Variability	4
2.4	Monetary Dependency	5
2.5	Fitting a Model on an Embedded System	5
2.5.1	What Quantization Does	6
2.5.2	Warm Context	6
2.5.3	Threading	6
2.6	Framing of Gap A	7
3	Gap B — Closed-Set Named-Node Output as a Structural Safety Boundary	7
3.1	Open-Vocabulary Risk	7
3.2	Post-Hoc Validation vs. Structural Prevention	7
3.3	Token-Level Constraint	8
3.4	Why Names Instead of Coordinates	8
3.5	The Annotation Graph as a Safety Boundary	9
3.6	Aliases and Spatial Relationships	9
4	Gap C — LLM Inference Inside a Native Nav2 BehaviorTree Node	9
4.1	BehaviorTree Basics	9
4.2	External LLM vs. Native Plugin Architecture	9
4.3	Plugin Integration	9
4.4	The Blackboard	10
4.5	The Seven-Step Tick Sequence	10
4.6	Comparison with BTGenBot	10
5	Gap D — Deployment-Log-Trained Confidence Calibration	10
5.1	Calibration vs. Accuracy	10
5.2	Deployment-Specific Calibration	11

5.3	The Five Calibration Features	11
5.4	Platt-Scaled Logistic Regression	11
5.5	Expected Calibration Error (ECE)	11
5.6	Safety Thresholds	11
6	Combining the Gaps	12
6.1	Gap B + Gap D	12
6.2	Gap B + Gap C	12
6.3	Gap A + Gap C	12
7	The Complete Architecture	12
7.1	What Each Layer Protects Against	13
7.2	Why All Four Are Needed	13
8	Two Failure Types	13
9	Defense in Depth	13
10	Positioning Against the Literature	14
11	Precise Claims for Written and Oral Defense	15

1 Understanding the Problem

Before discussing the four gaps individually, it is necessary to understand *what exactly the system is replacing*. A conventional Nav2 robot expects a metric pose, not a natural-language description of a place. Nav2 does not inherently understand the semantic concept “charging dock” — it understands a coordinate frame.

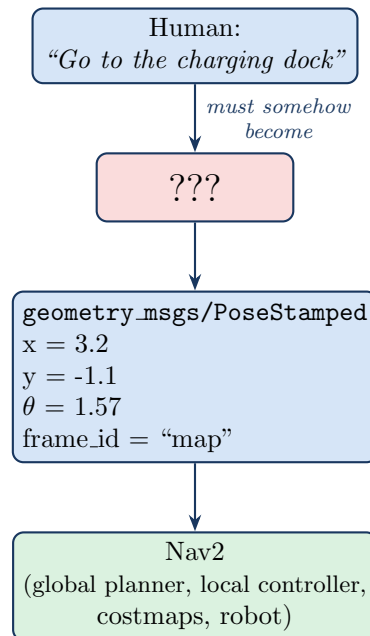


Figure 1: The core translation problem: Nav2 only understands metric poses, not named places.

SafeNav-LLM introduces a semantic layer that bridges this gap:

The LLM sits between a messy human language space and a physical robot control space, which is precisely why safety becomes the central engineering concern.

2 Gap A — Fully On-Device LLM Inference

2.1 Cloud-Based Architecture and Its Dependencies

A cloud-based architecture routes every command through the internet:

Listing 1: Example cloud response for a resolved room

```
{
  "room": "server_room",
  "confidence": 0.91
}
```

This convenience creates three coupled dependencies, illustrated below.

The design target environments — hospitals, factory floors, cleanrooms, and secure research facilities — may not permit outbound internet access at all. The requirement is therefore that *the robot must possess everything necessary to perform semantic resolution locally*.

2.2 Privacy Dependency

Even commands without personal names can leak sensitive facility information: building layout, room names, operational procedures, and restricted-area identifiers. Local inference eliminates

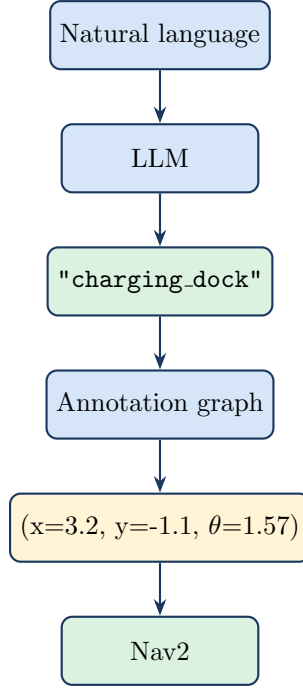


Figure 2: The semantic waypoint resolution pipeline introduced by SafeNav-LLM.

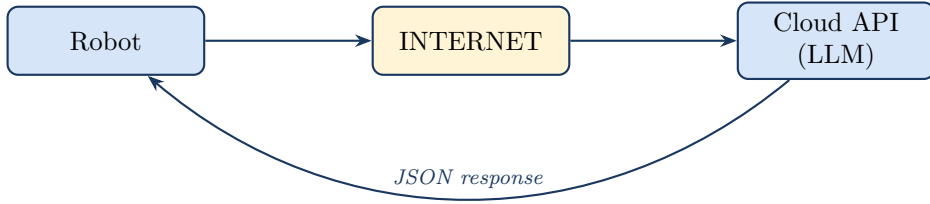


Figure 3: Cloud-based LLM resolution: convenient, but network-dependent.

this exposure entirely.

2.3 Latency Variability

Cloud inference latency is not simply the LLM’s compute time; it is the sum of network upload, server queueing, LLM inference, and network download:

$$\text{Total latency} = \underbrace{t_{\text{up}}}_{\text{network}} + \underbrace{t_{\text{queue}}}_{\text{server}} + \underbrace{t_{\text{LLM}}}_{\text{inference}} + \underbrace{t_{\text{down}}}_{\text{network}}$$

Request	Observed latency
Request 1	800 ms
Request 2	1.4 s
Request 3	3.7 s
Request 4	timeout

Table 1: Illustrative variability of cloud round-trip latency across requests.

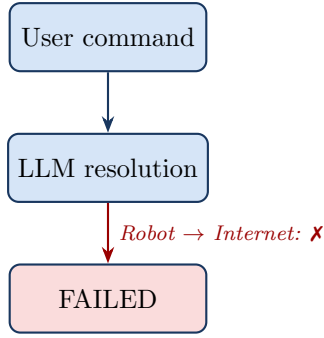


Figure 4: Connectivity dependency: no network means no semantic resolution.

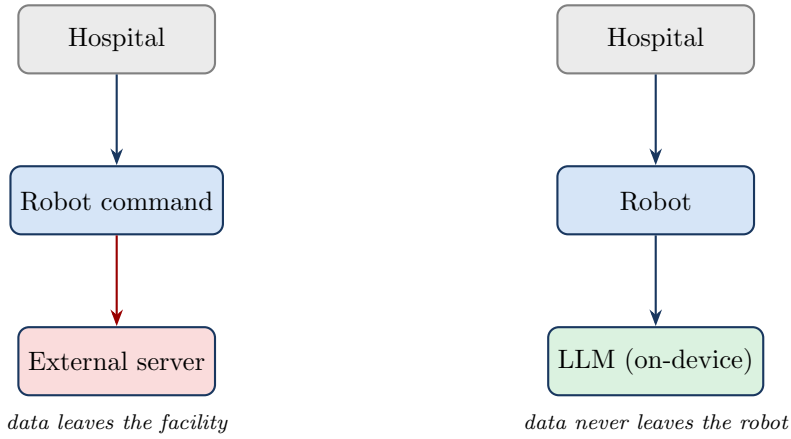


Figure 5: Cloud inference introduces a data-flow boundary outside the facility (left); local inference keeps commands entirely on the robot (right).

This unpredictability is particularly undesirable inside a BehaviorTree, which motivates a local model together with an explicit `timeout_ms` and a target p90 hardware latency of ≤ 3 seconds.

2.4 Monetary Dependency

Cloud LLM calls scale linearly with usage:

$$\text{Annual API calls} \approx 10 \frac{\text{cmds}}{\text{hr}} \times 8 \frac{\text{hr}}{\text{day}} \times 300 \text{ days} = 24,000 \text{ calls/year}$$

whereas on-device inference trades network/API cost for a one-time hardware cost plus electricity and maintenance. This is a practical, not purely scientific, deployment consideration.

2.5 Fitting a Model on an Embedded System

The robot’s Jetson Orin NX has finite RAM, VRAM/shared memory, compute, thermal budget, and power budget. A small model is therefore selected and quantized:

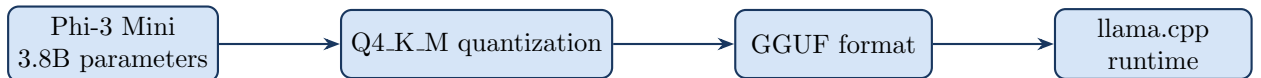


Figure 6: Model preparation pipeline: quantized and packaged for local inference (≈ 2.4 GB total footprint).

2.5.1 What Quantization Does

A neural network's weight tensor $W = [0.2387, -1.3928, 0.0027, \dots]$ is normally stored at high numerical precision. Quantization reduces the bits used per parameter:

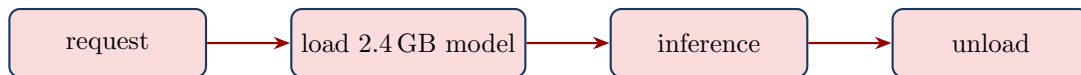


less memory + less computation $\Rightarrow$ potential quality loss

Figure 7: Progressive precision reduction. Q4_K_M is not a file-compression trick; it changes how inference is performed.

2.5.2 Warm Context

Naive (repeated per call)



SafeNav-LLM (warm context)



Figure 8: Loading the model once at startup eliminates per-call loading overhead.

2.5.3 Threading

Running inference directly on the ROS spin thread risks blocking other callbacks (localization, costmap updates, control) for hundreds of milliseconds. SafeNav-LLM instead dispatches inference to a dedicated `rc1cpp::ThreadPool` with two workers.

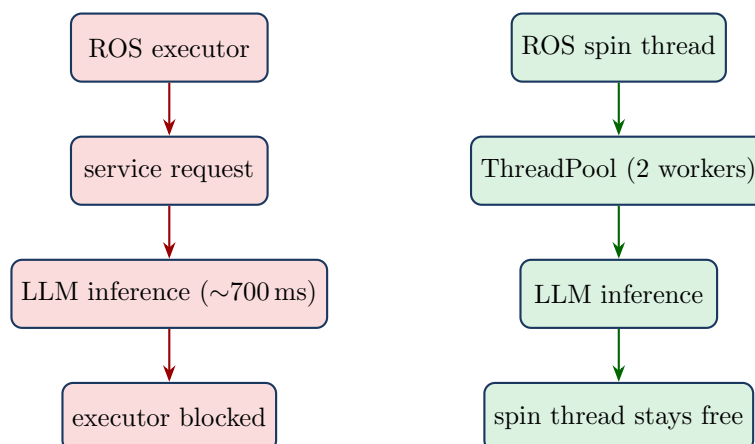


Figure 9: Blocking inference on the executor thread (left) versus offloading to a thread pool (right, as used in SafeNav-LLM).

2.6 Framing of Gap A

What Gap A actually means

The gap is *not* that nobody has run an LLM locally — prior work such as LocalNav already demonstrates on-device LLM/VLM inference. Rather: *existing work demonstrates that on-device inference is feasible, but does not combine it with the specific semantic way-point safety boundary, native Nav2 BT integration, and deployment-specific calibration of SafeNav-LLM.*

3 Gap B — Closed-Set Named-Node Output as a Structural Safety Boundary

3.1 Open-Vocabulary Risk

An ordinary LLM has a huge vocabulary and can generate room names that do not exist in the building, because language models know language — not the authoritative world state of a particular building.

Known rooms (graph)	Possible LLM output
Kitchen	Cafeteria
Server Room	Laboratory
Charging Dock	Warehouse
Reception	ICU, Storage Annex

Table 2: An unconstrained model can generate rooms outside the known set.

3.2 Post-Hoc Validation vs. Structural Prevention

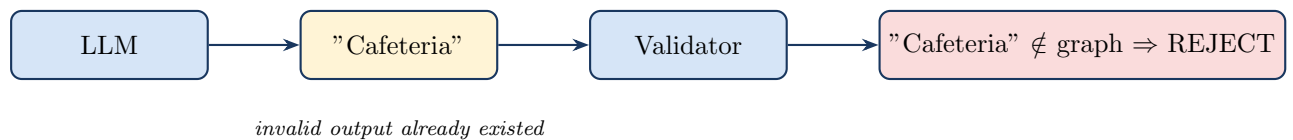


Figure 10: Post-hoc validation (e.g. as used in SPINE) rejects invalid output only *after* it has been generated.

SafeNav-LLM instead constrains the *generation grammar itself* so that invalid names are not merely rejected — they are not a reachable generation path:

Listing 2: GBNF-style structural constraint on room values

```

Allowed rooms = { kitchen, server_room, charging_dock, reception }

room-value ::=
  "kitchen"
| "server_room"
| "charging_dock"
| "reception"
  
```

3.3 Token-Level Constraint

LLMs generate text sequentially, producing a probability distribution $P(\text{token} \mid \text{previous tokens})$ at each step. A constrained decoder restricts the *support* of this distribution before sampling:

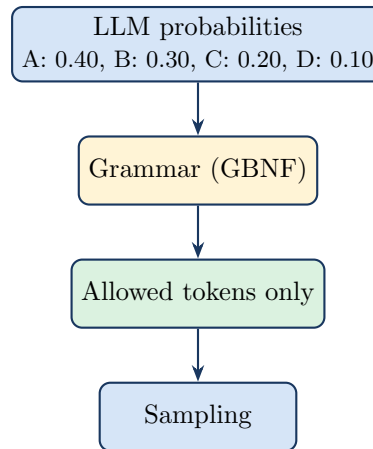


Figure 11: Grammar constraints act *during* generation, at the token-sampler level, rather than after a finished answer is produced.

3.4 Why Names Instead of Coordinates

Allowing the LLM to emit raw coordinates (x, y, θ) would make it participate in unconstrained geometry generation. Instead, the LLM only selects a semantic label; a deterministic lookup resolves the pose.

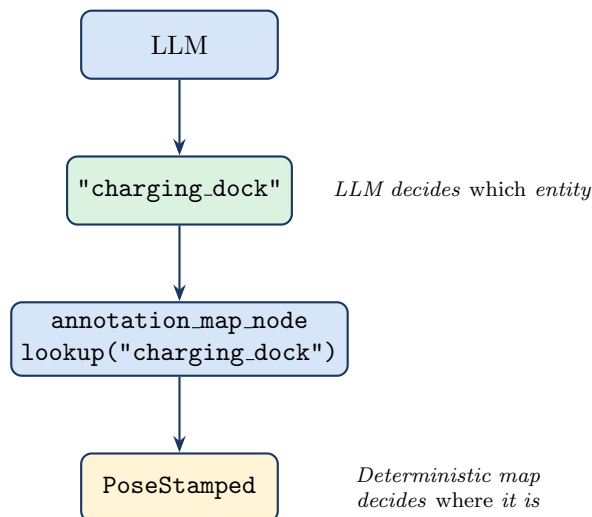


Figure 12: Separation of responsibilities: semantic selection (LLM) versus deterministic coordinate lookup (annotation graph).

Listing 3: Annotation graph entry: a deterministic name-to-pose mapping

```

{
  "name": "charging_dock",
  "aliases": ["charging station", "dock", "charger", "power point"],
  "pose": { "x": 3.2, "y": -1.1, "theta": 1.57, "frame": "map" }
}
  
```

3.5 The Annotation Graph as a Safety Boundary

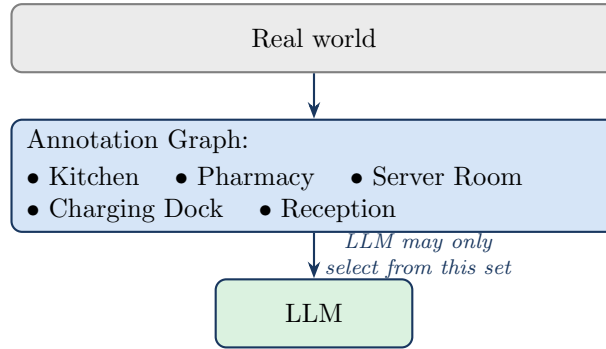


Figure 13: The annotation graph forms a bounded semantic universe between unbounded language generation and bounded robot action.

Does the closed set guarantee physical safety?

No. A closed-set constraint guarantees that the generated semantic location belongs to the validated set. It does *not* guarantee the robot cannot collide — the annotation itself could be stale, an obstacle could have moved, or the map could be outdated. Nav2’s planners, costmaps, and localization remain responsible for actual motion safety. The correct claim is narrower: *the closed set structurally prevents out-of-graph semantic/coordinate hallucination from the LLM* — one safety layer among several, not the entire safety system.

3.6 Aliases and Spatial Relationships

The graph stores linguistic variants that all resolve to one canonical node, and can encode spatial edges (e.g. *adjacent*) enabling relational commands such as “the room next to the charging dock.”



Figure 14: Spatial-relationship edges allow relational natural-language commands while keeping physical locations in the graph.

4 Gap C — LLM Inference Inside a Native Nav2 BehaviorTree Node

4.1 BehaviorTree Basics

A BehaviorTree is a hierarchical decision/execution structure. Nodes return one of **SUCCESS**, **FAILURE**, or **RUNNING**.

4.2 External LLM vs. Native Plugin Architecture

4.3 Plugin Integration

The plugin inherits from `BT::ActionNodeBase` and is loaded via `pluginlib` without modifying Nav2 source:

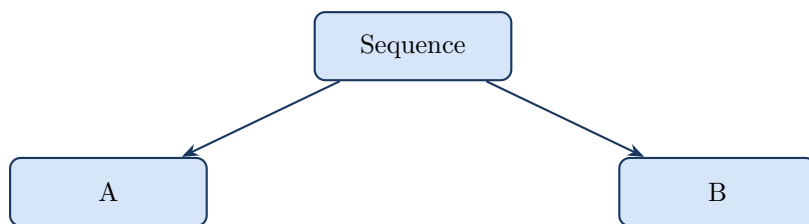


Figure 15: A **Sequence** executes A, then B only if A succeeds.

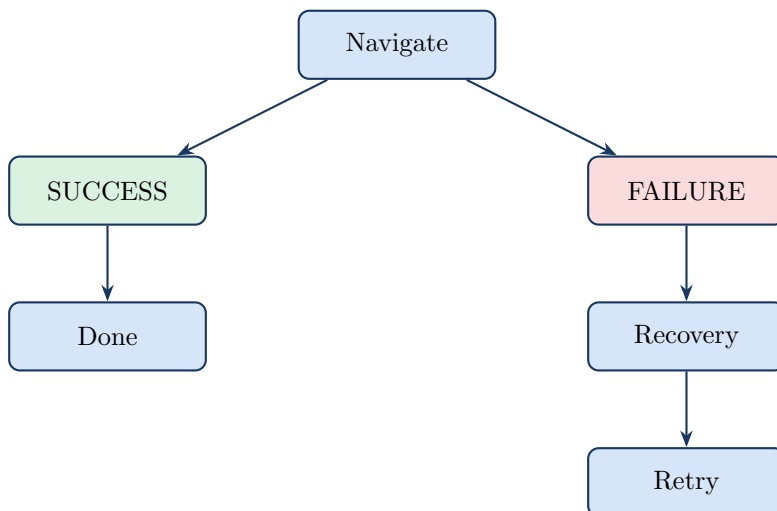


Figure 16: BehaviorTrees naturally express navigate–recover–retry mission logic.

Listing 4: The only Nav2 configuration change required

```

plugin_lib_names:
- semantic_waypoint_planner
  
```

Because the node participates natively in the tree, it gains: tick timing, status propagation (SUCCESS/FAILURE), blackboard read/write access, fallback-branch triggering, configurable timeouts, and direct control over mission continuation.

4.4 The Blackboard

4.5 The Seven-Step Tick Sequence

Why FAILURE is useful

When resolution confidence is low, the system does not force a guess. `SemanticNavigateToPose` returns **FAILURE**, the BT fallback subtree runs a **SpeakText** clarification node, and the tree becomes part of the *uncertainty-management mechanism* rather than a passive executor of possibly-wrong poses.

4.6 Comparison with BTGenBot

5 Gap D — Deployment-Log-Trained Confidence Calibration

5.1 Calibration vs. Accuracy

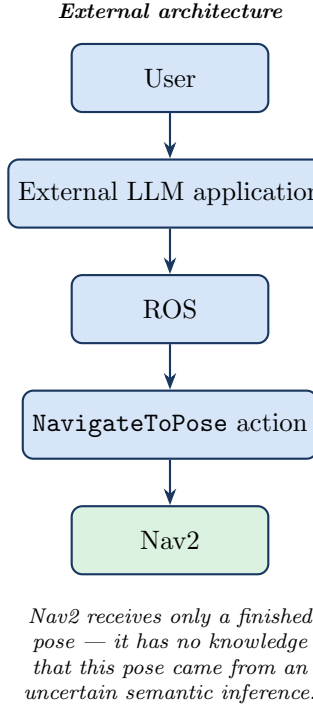


Figure 17: In an external-LLM architecture, semantic resolution happens entirely outside Nav2’s execution graph.

	Trials	Correct	Rate
Accuracy (raw)	100	85	85%
Confidence ≈ 0.9 (subset)	100	70	70% (overconfident)

Table 3: Accuracy asks how often the model is correct; calibration asks whether confidence values match actual correctness frequency.

5.2 Deployment-Specific Calibration

The same LLM, deployed in two different buildings, faces different uncertainty distributions: exact room names in one building versus vague relational phrasing (“the lab next to the server area”) in another. General LLM confidence therefore does not equal deployment-specific reliability. SafeNav-LLM collects $5 \text{ users} \times 40 \text{ commands} = 200$ trials from the actual deployment as its calibration training set.

5.3 The Five Calibration Features

5.4 Platt-Scaled Logistic Regression

$$\mathbf{x} = [\text{raw_confidence}, \text{entropy}, \text{candidate_count}, \text{command_length}, \text{edit_distance}]$$

$$P(\text{correct} \mid \mathbf{x}) = \sigma(w^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

5.5 Expected Calibration Error (ECE)

5.6 Safety Thresholds

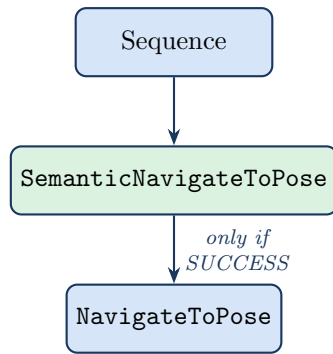


Figure 18: Native architecture: semantic resolution is itself a BehaviorTree action node, gating the rest of the tree.

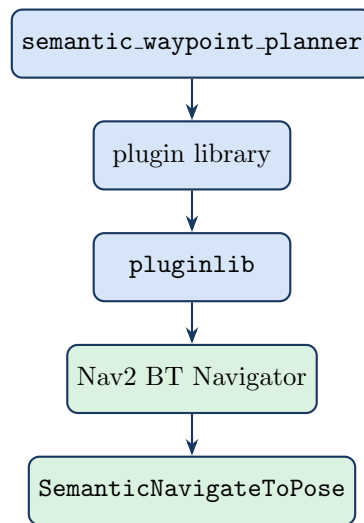


Figure 19: Loading path from the compiled plugin library into the running Nav2 BT Navigator.

6 Combining the Gaps

6.1 Gap B + Gap D

The closed set answers “*Is this a legitimate known location?*”; the calibrator answers “*Do we trust this particular interpretation?*” — two entirely different questions.

6.2 Gap B + Gap C

6.3 Gap A + Gap C

On-device inference means the BT can operate with a bounded, local timeout (3000ms for the resolver; 5000ms default for the broader BT timing constraint) without depending on an external service.

7 The Complete Architecture

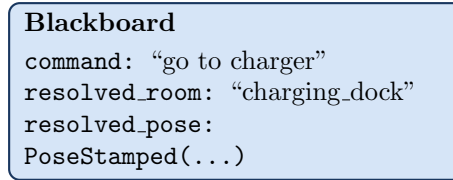


Figure 20: Shared state written by the semantic node and consumed by downstream BT nodes.

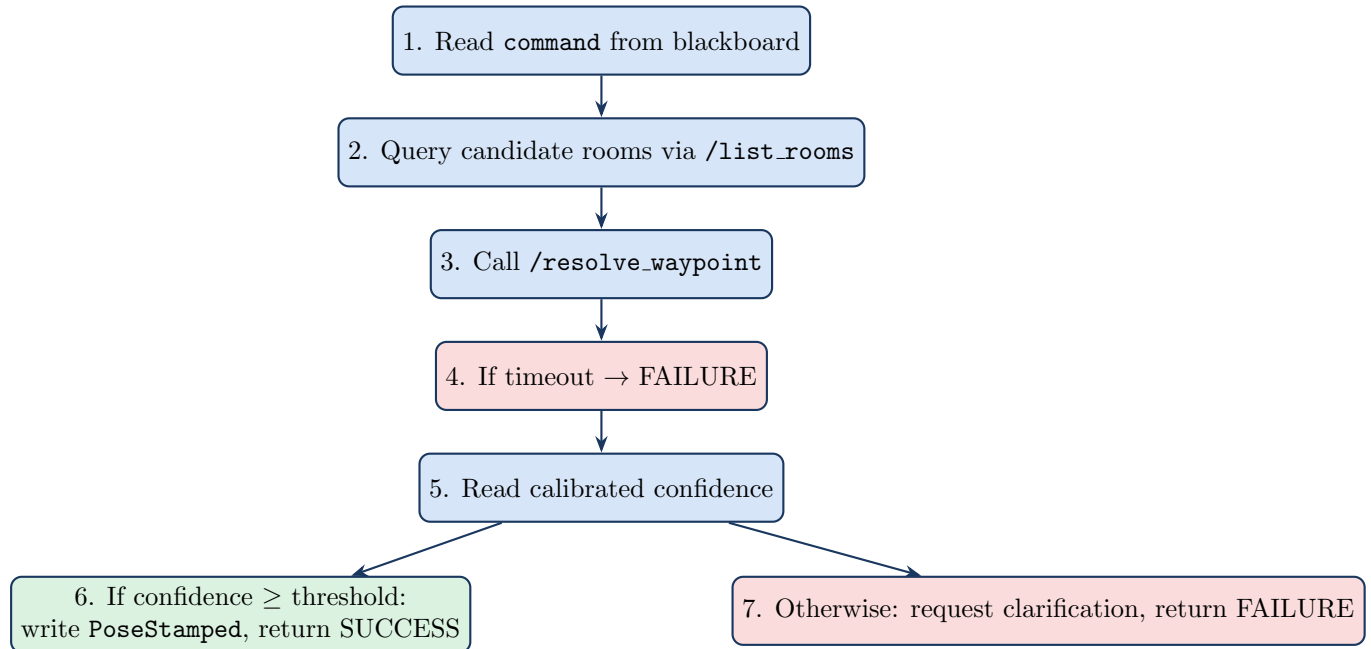


Figure 21: The `SemanticNavigateToPose` tick logic, as specified in the Phase 2 design.

7.1 What Each Layer Protects Against

The four questions

Gap A: Can the robot think locally?

Gap B: Can it be prevented from inventing places?

Gap C: Can that reasoning become a native part of robot execution?

Gap D: Can the robot know when it should *not* trust its interpretation?

7.2 Why All Four Are Needed

8 Two Failure Types

9 Defense in Depth

Gap A controls *where* computation happens.

Gap B controls *what* the model is allowed to output.

Gap D controls *whether* the output is trusted.

Gap C controls *how* the robot acts on that decision.

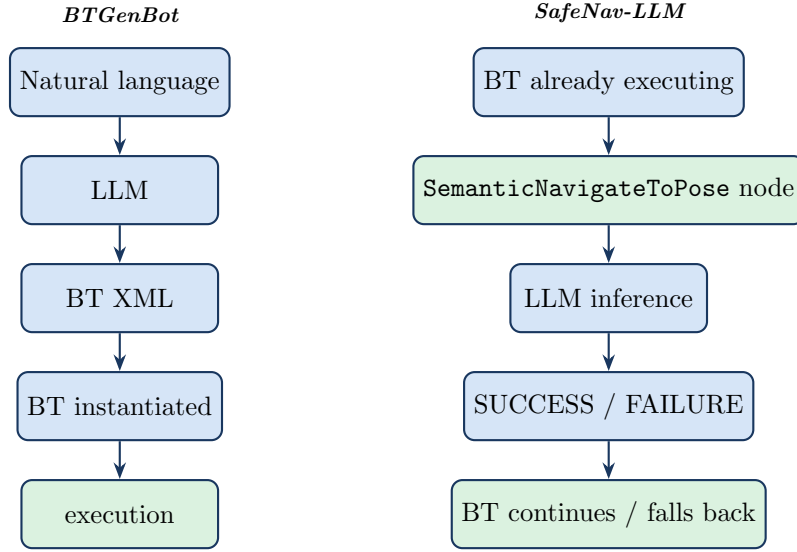


Figure 22: BTGenBot’s LLM generates a tree *before* execution; SafeNav-LLM’s LLM runs *inside* a live tree action node.

Feature	Description
Raw LLM confidence	The model’s self-reported score, e.g. 0.91 — learned to be predictive or not.
Token entropy	Concentration of the output distribution; low entropy (e.g. 0.95/0.02/0.02/0.01) signals confident selection, high entropy (e.g. 0.35/0.30/0.20/0.15) signals ambiguity.
Candidate room count	Size of the semantic search space (e.g. 3 rooms vs. 30 rooms).
Command length	Longer, more structurally complex commands can correlate with resolution difficulty.
Levenshtein distance	Minimum edit distance between the resolved room name and a substring of the command — captures lexical similarity (or its absence).

Table 4: The five deployment features used by the calibrator.

10 Positioning Against the Literature

The precise novelty claim

osmAG-LLM already restricts an LLM to named semantic nodes, so SafeNav-LLM cannot claim that restriction as novel in isolation. The stronger claim is the *combination*:

osmAG-like semantic restriction + token-level grammar enforcement + explicit safety-boundary framing + on-device inference + native Nav2 BT pluginlib node + deployment-log calibration.

The contribution is not that any individual component — local LLMs, closed-set semantic navigation, BehaviorTrees, or calibration — is unprecedented. The research gap is the absence, in the reviewed literature, of a system that combines all four in a single native Nav2 architecture.

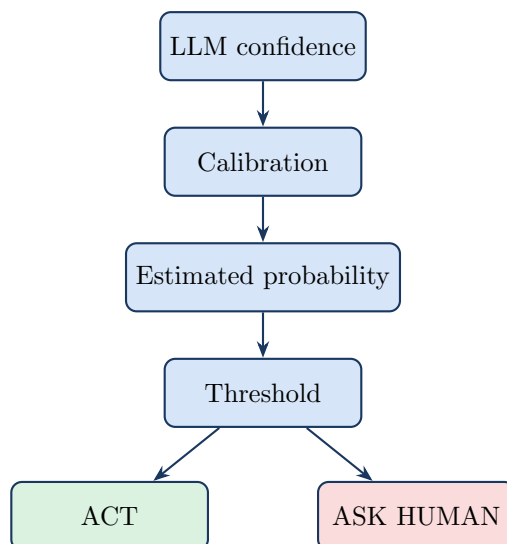


Figure 23: Calibration converts a vague confidence number into an operational safety decision.

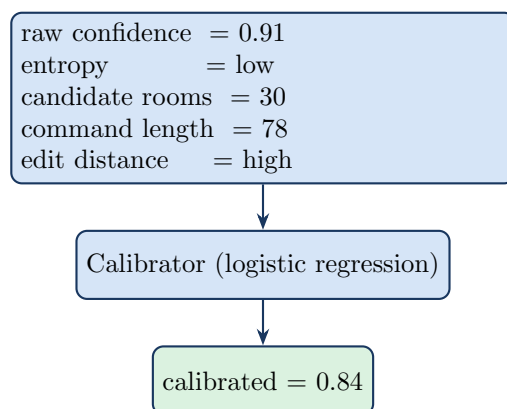


Figure 24: Feature vector combined via Platt-scaled logistic regression into a calibrated probability, which is far more useful as a decision variable than the raw 0.91.

11 Precise Claims for Written and Oral Defense

Recommended phrasing

Avoid: “Our system prevents the robot from going to an unsafe location.” (*too broad*)

Prefer:

“Our closed-set constraint prevents the language model from generating a semantic waypoint outside the pre-validated annotation graph, while the calibrated confidence gate prevents sufficiently uncertain in-graph resolutions from being passed to Nav2.”

“The Nav2 BehaviorTree plugin provides the execution-level mechanism that converts acceptance into navigation and rejection into clarification.”

“The contribution is not that any individual component — local LLMs, closed-set semantic navigation, BehaviorTrees, or calibration — is unprecedented. The research gap is the absence, according to the reviewed literature, of a system that combines all four in a single native Nav2 architecture.”

Confidence bin	Actual accuracy
0.5–0.6	0.54
0.6–0.7	0.68
0.7–0.8	0.75
0.8–0.9	0.84
0.9–1.0	0.91

Table 5: Well-calibrated confidence bins track actual accuracy closely. Target: $\text{ECE} \leq 0.08$.

Error type	Target	Consequence
False pass (SAFE but wrong)	$\leq 5\%$	Dangerous — robot acts incorrectly
False rejection (UNSAFE but correct)	$\leq 15\%$	Safer, but annoying — asks human unnecessarily

Table 6: Deliberate asymmetric safety tradeoff: rejections are tolerated far more than false passes.

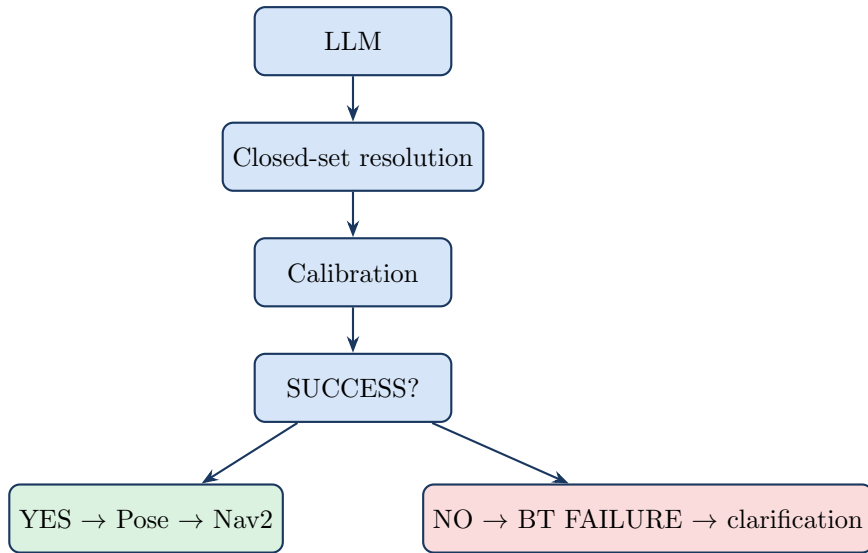


Figure 25: Uncertainty becomes explicit BehaviorTree control flow rather than a silent error.

Layer	Main problem it prevents
On-device inference (Gap A)	Cloud/network dependency
Closed-set decoding (Gap B)	Invented / nonexistent locations
Native BT plugin (Gap C)	AI logic disconnected from Nav2’s execution semantics
Calibration model (Gap D)	Overconfident but incorrect semantic resolutions

Table 7: Summary role of each architectural gap.

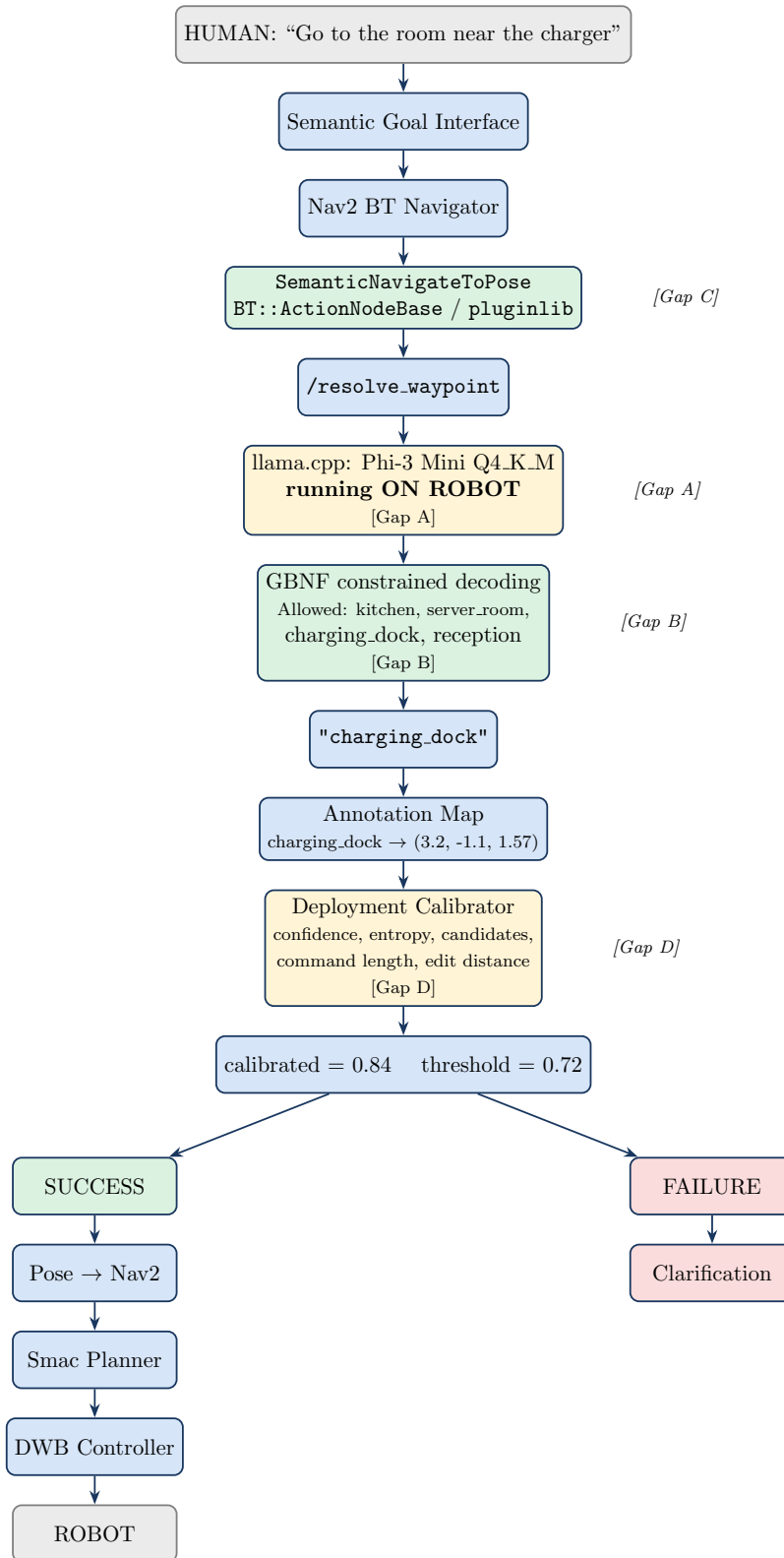
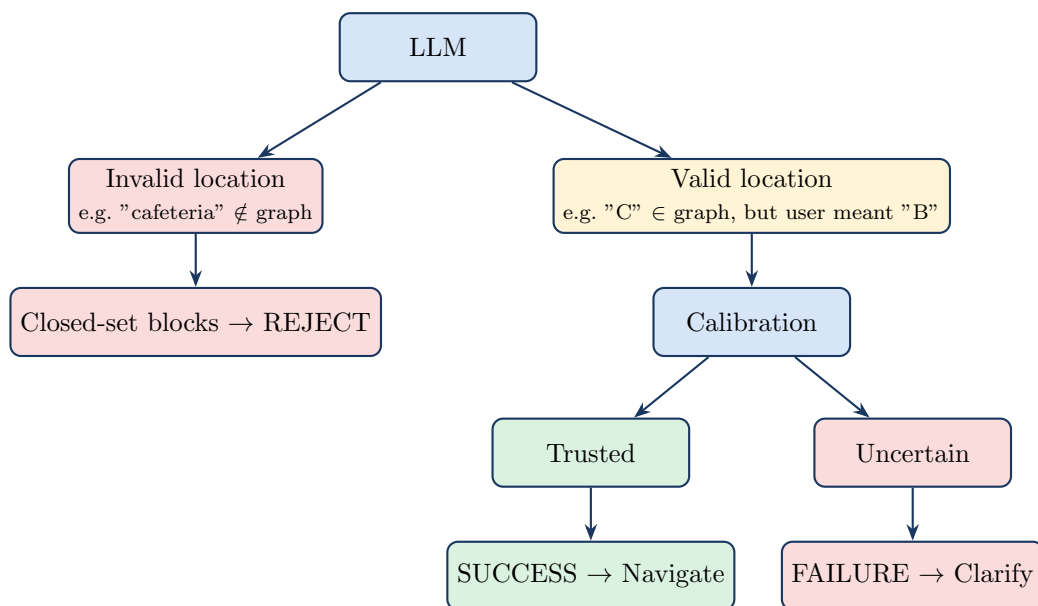


Figure 26: The complete SafeNav-LLM pipeline, integrating all four architectural gaps.

Removed	Consequence
Gap A	Safe semantic resolver, but the robot cannot function offline.
Gap B	LLM may still generate an invalid entity (e.g. “cafeteria”); calibration alone cannot prevent an invalid entity from being generated.
Gap C	Semantic system lives outside Nav2; native BT status propagation, blackboard integration, and fallback trees are lost.
Gap D	A wrong <i>but valid</i> room can still be executed — the closed-set constraint cannot catch semantically valid misinterpretation.

Table 8: Removing any single gap reintroduces a distinct failure mode.

Figure 27: Two distinct failure types require two distinct solutions: closed-set constraint prevents *invalid-world hallucination*; calibration handles *valid-world misinterpretation*.

	Failure Type 1	Failure Type 2
Name	Invalid-world hallucination	Valid-world misinterpretation
Example	User says “cafeteria”; LLM outputs “cafeteria” ∉ graph	User means room B; LLM confidently outputs room C, which exists
Caught by closed set?	Yes	No — C is a legitimate node
Solution	Closed-set constrained decoding (Gap B)	Confidence calibration + thresholding (Gap D)

Table 9: Why both Gap B and Gap D are independently necessary.

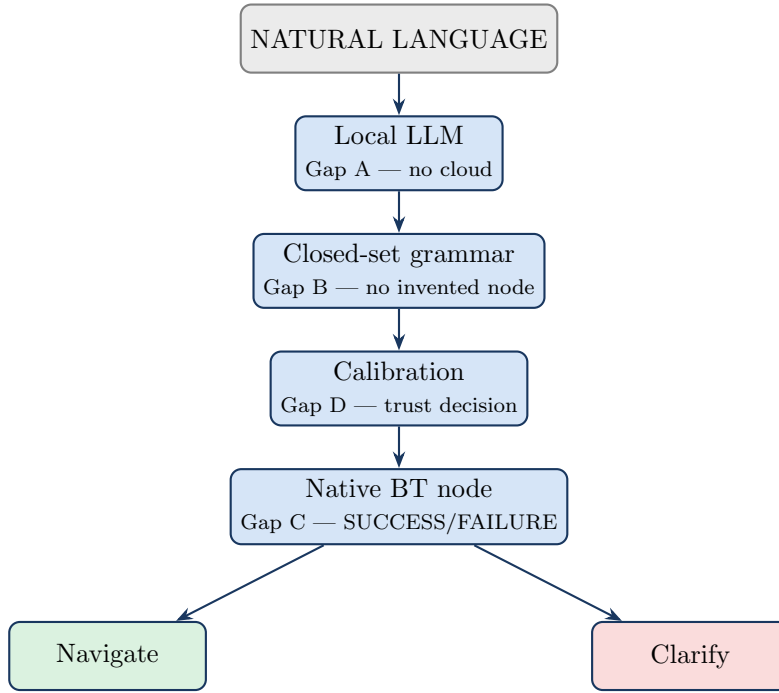


Figure 28: The four gaps as sequential layers of defense, each with a distinct responsibility.

System	Contribution	Gap relative to SafeNav-LLM
LM-Nav	LLM + navigation + graph-based restriction	Cloud GPT-3; no deployment calibration; not Nav2-native; outdoor focus
SayCan	LLM + closed skill set + physical feasibility	Cloud PaLM; skill repertoire rather than annotation graph; manipulation focus
NLMap	Open-vocabulary semantic navigation	Argues <i>against</i> a closed set — opposite safety philosophy
Das et al.	LLM + ROS2 + Nav2 navigation	Cloud models; free-form intent; no calibration; external Nav2 wrapper
BTGenBot	LLM + BehaviorTree + Jetson + Nav2	LLM generates BT XML <i>before</i> execution, not embedded in a running plugin node
KnowNo	Uncertainty + human clarification	Conformal prediction, not deployment-log-trained supervised calibration
SPINE	LLM navigation + safety validation	Safety mechanism is post-hoc, not generation-time constrained
LocalNav	On-device model on Jetson + navigation	No closed-set/Nav2-BT/calibration combination
osmAG-LLM	Named semantic nodes, no coordinates, constrained output	Most structurally similar prior system — missing BT/pluginlib integration and deployment calibration

Table 10: Positioning of SafeNav-LLM relative to closely related prior systems.